MERN Stack Internship

DAY 10 ASSIGNMENT REPORT

Name: CHAYAN SONI
Course: BTech CSIT 1 year
Position : MERNStackINTERN

Allocated Date:10/06/26
Submission Date: 10/006/26

MERN Stack Internship – Day 10 Report

Project Title
Employee Attendance Dashboard – Component Architecture Optimization & State Management

---

Objective
The objective of this task was to build and optimize an Employee Attendance Dashboard using React. The project focused on centralized state management using Context API, identifying and preventing unnecessary component re-renders using React.memo, understanding the lifecycle of useEffect, and explaining how React's Virtual DOM improves application performance.

Technologies Used

- React.js
- JavaScript (ES6)
- CSS3
- React Hooks (useState, useEffect, useContext, useMemo)
- React Context API
- React.memo
- JSON Data Source

Project Overview
The Employee Attendance Dashboard displays employee information and salary statistics through reusable React components.

The application consists of:

- Dashboard Component
- EmployeeList Component
- SalaryChart Component

- EmployeeContext Provider
- JSON Data Source

Features Implemented

1. Employee Data Management

Employee records were stored inside a JSON file and loaded into React state.

Sample Data:

- Rahul (IT)
- Sneha (HR)
- Aman (Finance)
- Priya (Marketing)

---

2. Employee List Component

The EmployeeList component displays:

- Employee Name
- Department
- Salary

The component dynamically renders employee records using the map() method.

---

3. Salary Summary Component

The SalaryChart component calculates and displays:

- Total Employee Salary

The calculation is optimized using useMemo() to prevent unnecessary recalculations.

---

4. Context API Integration

A centralized EmployeeContext was created to manage employee data.

Benefits:

- Eliminated prop drilling
- Centralized state management
- Improved component communication

Implementation:

- createContext()
- Provider
- useContext()

---

5. React.memo Optimization

Both EmployeeList and SalaryChart components were wrapped using React.memo().

Benefits:

- Prevents unnecessary re-renders
- Improves rendering efficiency
- Enhances application performance

---

6. useEffect Lifecycle Demonstration

Lifecycle stages demonstrated:

Mounting Phase

Executed when component loads.

Example:

- Dashboard Mounted

Updating Phase

Executed whenever employee state changes.

Example:

- Employees Updated

Unmounting Phase

Cleanup function executed when component is removed.

Example:

- Dashboard Unmounted

---

7. Virtual DOM Concept

React uses a Virtual DOM to improve rendering performance.

Process:

1. Create Virtual DOM copy
2. Compare old and new Virtual DOM
3. Identify changed elements
4. Update only required DOM nodes

Benefits:

- Faster rendering
- Reduced DOM manipulation
- Better user experience

Component Structure

```
src/
├─── App.jsx
├─── EmployeeList.jsx
├─── SalaryChart.jsx
├─── context/
│   └─── EmployeeContext.jsx
├─── data/
│   └─── employees.json
```

├──── App.css

└──── main.jsx

Optimization Techniques Applied

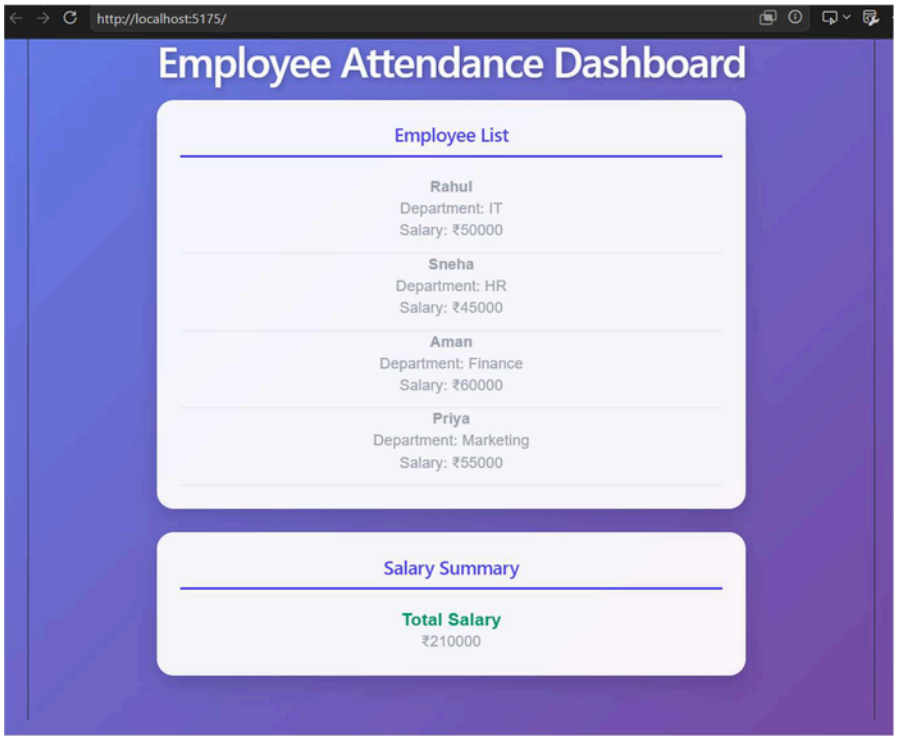
Optimization	Purpose
React.memo	Prevent unnecessary renders
Context API	Centralized state management
useMemo	Optimize salary calculations
useEffect	Lifecycle management
Virtual DOM	Efficient UI updates

Learning Outcomes

Through this project I learned:

- □ React Context API
- □ Centralized State Management
- □ Component Optimization Techniques
- □ React.memo Usage
- □ useMemo Optimization
- □ useEffect Lifecycle Phases
- □ Virtual DOM Working Mechanism
- □ React Rendering Behavior
- □ Performance Optimization Strategies

Output Screenshots



[

```
import { createContext } from "react";

export const EmployeeContext = createContext();
```

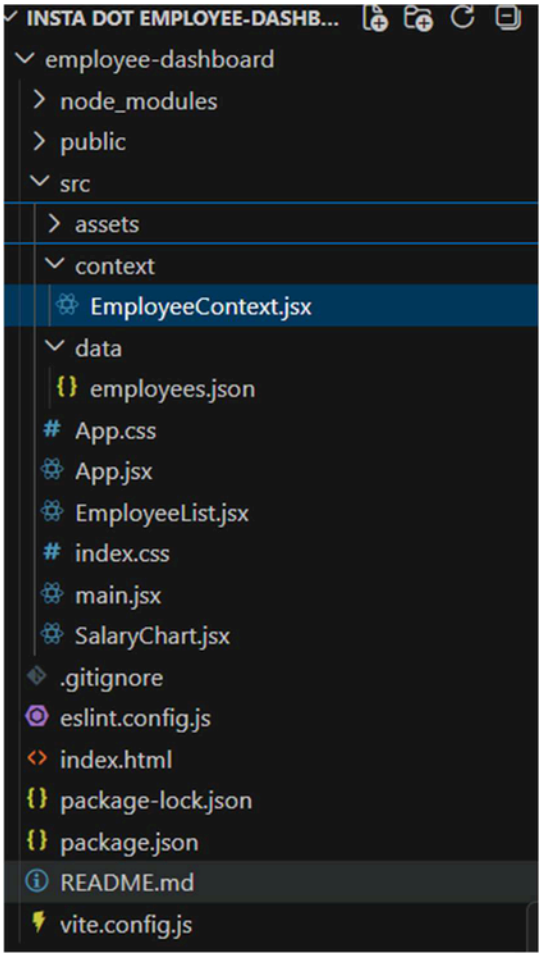
employee-dashboard > src > SalaryChart.jsx > SalaryChart > memo() callback

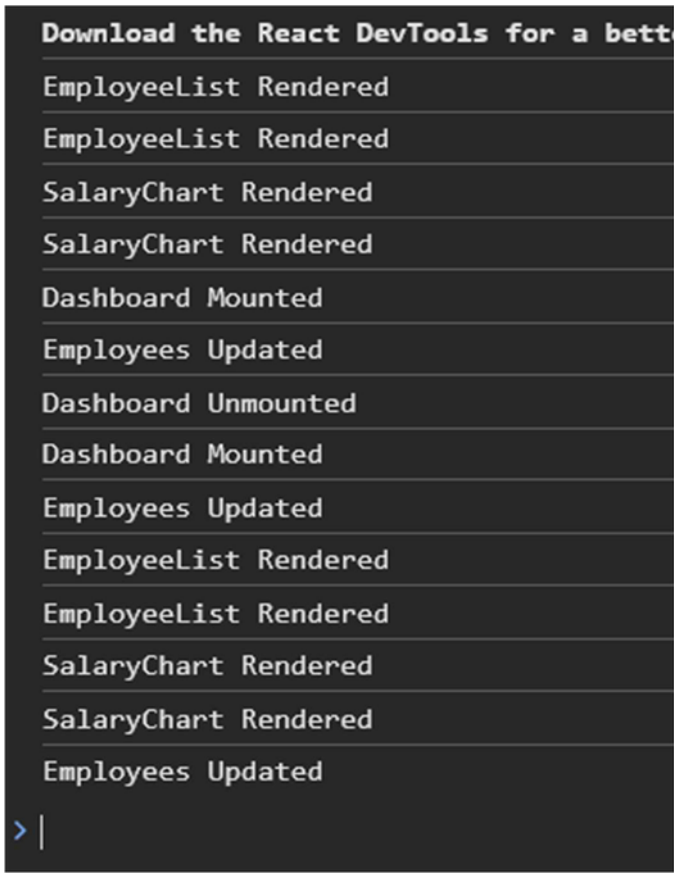
C:\Users\yashm\Downloads\insta dot employee-dashboard\employee-dashboard

```
import { EmployeeContext } from "./context/EmployeeContext";

const SalaryChart = memo(() => {
  const { employees } = useContext(EmployeeContext);

  console.log("SalaryChart Rendered");

  const totalSalary = useMemo(() => {
    return employees.reduce((sum, emp) => sum + emp.salary, 0);
  }, [employees]);

  return (
    <div className="card">
      <h2>Salary Summary</h2>

      <h3>Total Salary</h3>

      <p>₹{totalSalary}</p>
    </div>
  );
});

export default SalaryChart;
```

```
employee-dashboard > src > EmployeeList.jsx > default
```

```jsx
import React, { memo, useContext } from "react";
import { EmployeeContext } from "./context/EmployeeContext";

const EmployeeList = memo(() => {
  const { employees } = useContext(EmployeeContext);

  console.log("EmployeeList Rendered");

  return (
    <div className="card">
      <h2>Employee List</h2>

      {employees.map((emp) => (
        <div key={emp.id}>
          <p>
            <strong>{emp.name}</strong>
          </p>

          <p>Department: {emp.department}</p>

          <p>Salary: ₹{emp.salary}</p>

          <hr />
        </div>
      ))}
    </div>
  );
});

export default EmployeeList;
```





Conclusion

The Employee Attendance Dashboard was successfully developed and optimized using React best practices. The application demonstrates centralized state management through Context API, efficient rendering using React.memo and useMemo, lifecycle handling using useEffect, and improved performance through React's Virtual DOM architecture. The project strengthened understanding of React component architecture, state flow, and performance optimization techniques.